# AI Accelerator Design Project Rubric

## Overview

This project will evaluate you  on your ability to **design, implement, and evaluate a synthesizable AI accelerator using SystemC and high-level synthesis**, targeting Amazon EC2 F2 FPGAs. Successful projects tackle a complex ML workload and demonstrate **architectural insight, FPGA-aware design discipline, correct SystemC implementation, a usable programming model, and credible evaluation based on synthesis results**.

The emphasis is on **executable microarchitecture *and* a realistic software–hardware interface**.

## 1. Problem Definition & FPGA Motivation (10%)

Projects should clearly define:

- The target ML workload (kernel(s), model fragment, tensor shapes, datatypes).
- The performance objective (throughput, latency, efficiency).
- Why an FPGA is a good match for this workload.

Strong projects articulate **FPGA-specific advantages** (streaming pipelines, low latency, custom precision, fixed-function datapaths), rather than generic acceleration arguments.

Up to 10% extra credit for more complex ML workloads

## 2. Workload Characterization & Baseline (10%)

You must demonstrate understanding of the workload *prior* to hardware mapping. This includes:

- Quantifying computation, data movement, and reuse.
- Identifying the dominant bottleneck.
- Providing a reasonable CPU or GPU baseline.
- Justifying the partitioning between host code and FPGA kernels.

Strong projects show that **the accelerator structure follows directly from workload properties**.

Up to 10% extra credit for thorough analysis of a complex ML workload.

## 3. SystemC Microarchitecture & Functional Correctness (20%)

This category evaluates the **quality, structure, and correctness of the SystemC design**.

Projects should:

- Implement a clear microarchitecture (PEs, buffers, control logic).
- Use modular and readable SystemC code.

- Be functionally correct and validated against a golden reference.

## 4. Programming Model & Compiler Mapping (10%)

Projects should present a **clear programming model that explains how software targets the accelerator**.
This includes:

- The host-side API or kernel invocation interface.
- How ML operations (e.g., matmul, convolution, attention blocks) map onto one or more SystemC kernels.
- The role of compile-time vs run-time parameters (shapes, tiling, precision).
- Constraints imposed by the accelerator structure on software (e.g., fixed tile sizes, streaming order).

Strong projects make it clear that the accelerator is **usable**, and explain how design decisions simplify or complicate the mapping.

## 5. HLS-Aware Design & Optimization (20%)

You are expected to treat HLS as an **architectural design space**
Strong projects:

- Use HLS pragmas intentionally (pipeline, unroll, array partition).
- Analyze initiation interval (II), latency, and throughput.
- Explain tradeoffs between parallelism, resource usage, and clock frequency.
- Reference HLS reports to justify decisions.

## 6. Memory System & Data Movement (20%)

Because FPGA accelerators are often **memory- and buffering-limited**, this category is critical.
Projects should:

- Explicitly describe host → DRAM → on-chip → compute data movement.
- Use AXI interfaces and burst access appropriately.
- Design on-chip buffering (BRAM/URAM) to exploit reuse.
- Analyze bandwidth relative to F2 limits.

Strong projects treat **data movement as a first-class design problem**.

## 7. FPGA Resource Utilization & Timing (10%)

This category evaluates whether the design is **realistic on real hardware**.
Students should:

- Report post-HLS resource usage (DSPs, LUTs, BRAMs).
- Discuss achieved or target clock frequency.
- Explain what limits scaling (routing, DSPs, memory ports).

Ignoring resource or timing constraints indicates an incomplete design.

## 8. Evaluation, Results & Communication (10%)

You should support claims with evidence and communicate clearly.
This includes:

- Performance results from synthesis or simulation.
- Comparison to baseline or alternative mappings.
- Clear explanation of design tradeoffs and limitations.

Strong projects describe interesting engineering trade-offs not just numbers.